

Abstract Class

Interview Questions • Concepts • Tricky Questions • Coding • SDET Examples

01	Abstract Class Fundamentals
02	Instantiation & Constructors
03	Abstract Methods
04	Inheritance & Interfaces
05	Runtime Polymorphism
06	Tricky Interview Questions
07	Abstract Class vs Interface
08	SDET / Selenium Framework Example
09	Top 10 Questions for Revision

1. Abstract Class Fundamentals

Q1. What is an abstract class?

An abstract class is declared using the **abstract** keyword. It can contain abstract methods as well as concrete methods, variables and constructors. It is commonly used when related child classes share common behaviour but need their own specific implementation.

Q2. Why do we use an abstract class?

To provide common functionality to child classes while forcing subclasses to implement required behaviour through abstract methods. It is also useful when the base class should not be instantiated directly.

Q3. How do you declare an abstract class?

Use the **abstract** keyword before the class declaration.

```
abstract class Animal {  
}
```

Q4. Can an abstract class contain concrete methods?

Yes. It can contain both abstract methods and fully implemented concrete methods.

Q5. Can an abstract class contain variables?

Yes. It can contain instance variables, static variables and final variables.

2. Instantiation & Constructors

Q6. Can we create an object of an abstract class?

No. An abstract class cannot be instantiated directly.

```
abstract class Animal {  
}
```

```
Animal a = new Animal(); // Compilation error
```

Q7. Can we create a reference variable of an abstract class?

Yes. An abstract-class reference can point to an object of a concrete child class.

```
Animal animal = new Dog();
```

Q8. Can an abstract class have a constructor?

Yes. Its constructor is executed when a concrete subclass object is created.

```
abstract class Animal {  
  
    Animal() {  
        System.out.println("Animal constructor");  
    }  
  
    abstract void sound();  
}  
  
class Dog extends Animal {  
  
    Dog() {  
        System.out.println("Dog constructor");  
    }  
  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```

}

public class Test {

    public static void main(String[] args) {
        Animal animal = new Dog();
    }
}

```

Q9. Can an abstract class have multiple constructors?

Yes. Constructor overloading is allowed.

Q10. Can a constructor itself be abstract?

No. Constructors cannot be abstract because constructors are not inherited or overridden.

3. Abstract Methods

Q11. What is an abstract method?

An abstract method is declared without an implementation/body. It specifies behaviour that a concrete subclass must implement.

```

abstract void sound();

```

Q12. Can an abstract method have a body?

No. An abstract method does not have a method body.

Q13. Can a class contain an abstract method without being abstract?

No. A class containing an abstract method must itself be abstract.

Q14. What happens if a child class does not implement all abstract methods?

The child class must also be declared abstract. A concrete child class must implement all inherited abstract methods.

Q15. Can an abstract method be private?

No. A private method cannot be overridden, while an abstract method is intended to be implemented by a subclass.

Q16. Can an abstract method be final?

No. **abstract** requires overriding, while **final** prevents overriding.

Q17. Can an abstract method be static?

No. Static methods belong to the class and are not overridden in the runtime-polymorphism sense required by abstract methods.

4. Inheritance & Interfaces

Q18. Can an abstract class extend another abstract class?

Yes.

```

abstract class A {
    abstract void test();
}

abstract class B extends A {
}

```

Q19. Can an abstract class extend a concrete class?

Yes.

Q20. Can a concrete class extend an abstract class?

Yes. It must implement all inherited abstract methods.

Q21. Can an abstract class implement an interface?

Yes.

```
abstract class Test implements Runnable {  
}
```

Q22. Can an abstract class implement an interface without implementing all methods?

Yes. Because the class itself is abstract, it can leave interface methods for a concrete subclass.

Q23. Can an abstract class extend multiple classes?

No. Java does not support multiple class inheritance.

Q24. Can an abstract class implement multiple interfaces?

Yes.

```
abstract class Test implements A, B, C {  
}
```

5. Runtime Polymorphism

Q25. Can an abstract class support runtime polymorphism?

Yes. A parent abstract-class reference can refer to different child-class objects, and the overridden method implementation is selected at runtime.

```
abstract class Animal {  
    abstract void sound();  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog");  
    }  
}  
  
class Cat extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Cat");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
  
        Animal animal;  
  
        animal = new Dog();  
        animal.sound();  
  
        animal = new Cat();  
        animal.sound();  
    }  
}
```

Q26. Why is this runtime polymorphism?

The reference type is **Animal**, but the actual object can be **Dog** or **Cat**. The overridden method is resolved according to the actual object at runtime.

6. Tricky Interview Questions

Q27. Can an abstract class have zero abstract methods?

Yes. Declaring it abstract prevents direct instantiation.

```
abstract class Test {  
  
    void display() {  
        System.out.println("Hello");  
    }  
}
```

Q28. Can an abstract class have a final method?

Yes. A final method can be inherited but cannot be overridden.

Q29. Can an abstract class have a static method?

Yes.

Q30. Can an abstract class have a final variable?

Yes.

Q31. Can an abstract class have overloaded methods?

Yes.

Q32. Can an abstract class have a main() method?

Yes. **main()** is static and does not require an instance of the abstract class.

```
abstract class Test {  
  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

Q33. Can we create an object of an abstract class using an anonymous class?

Yes. The object is an instance of an anonymous concrete subclass.

```
abstract class Animal {  
  
    abstract void sound();  
}  
  
Animal animal = new Animal() {  
  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
};  
  
animal.sound();
```

7. Abstract Class vs Interface

Feature	Abstract Class	Interface
Declaration	abstract class	interface
Constructor	Yes	No

Instance variables	Yes	No instance state
Abstract methods	Yes	Yes
Concrete methods	Yes	default/static methods are supported
Inheritance	A class extends one class	A class can implement multiple interfaces
Purpose	Shared base + partial implementation	Contract / capability

Q34. When would you choose an abstract class over an interface?

When related classes need shared state, constructors, reusable implementation and common base behaviour, an abstract class can be appropriate. An interface is useful when the primary requirement is to define a contract or capability.

8. SDET / Selenium Framework Example

Q35. How can an abstract class be used in a Selenium Page Object framework?

A common approach is to create an abstract **BasePage** containing shared WebDriver functionality. Concrete pages such as LoginPage and HomePage extend it and implement page-specific behaviour.

```
abstract class BasePage {

    protected WebDriver driver;

    BasePage(WebDriver driver) {
        this.driver = driver;
    }

    abstract void verifyPage();

    void click(By locator) {
        driver.findElement(locator).click();
    }
}

class LoginPage extends BasePage {

    LoginPage(WebDriver driver) {
        super(driver);
    }

    @Override
    void verifyPage() {
        System.out.println("Login page verified");
    }
}
```

Q36. Why make BasePage abstract?

It communicates that BasePage is intended as a common parent rather than a page that should normally be instantiated directly. It can also force concrete pages to provide required page-specific implementations.

Q37. Can BasePage be abstract without having an abstract method?

Yes. The abstract modifier can prevent direct instantiation while still providing reusable common functionality.

9. Top 10 — Quick Revision

#	Question	Short Answer
---	----------	--------------

1	What is an abstract class?	A non-instantiable class that can contain abstract and concrete members.
2	Can we instantiate it?	No, not directly.
3	Can it have a constructor?	Yes.
4	Can it have zero abstract methods?	Yes.
5	Can it have concrete methods?	Yes.
6	Can an abstract method be final/private/static?	No; these conflict with required implementation/overriding behaviour.
7	Can it implement interfaces?	Yes, including multiple interfaces.
8	Can it extend another abstract class?	Yes.
9	Can it support runtime polymorphism?	Yes.
10	How is it useful in Selenium?	A BasePage can hold common WebDriver functionality while concrete pages implement

Interview Tip: For senior SDET interviews, do not stop at the definition. Explain why you would use an abstract class, give a short Java example, and relate it to a framework such as BasePage in Selenium.